

Unit 1: Introduction to Time Series & Tidyverse

Dr Christian Engels
christian_engels@outlook.com

Applied Timeseries in R
7 Feb 2026

Certificate in Engagement with Practice in Finance (CEPF)
Durham University Business School



What is a Time Series?

Tidyverse Essentials

Accessing Financial Data

Time Series Objects in R

Log Returns & Visualisation

Reshaping Data

Exercises

By the end of this unit you will be able to:

- 1 Define a time series and describe its structural components
- 2 Use **dplyr** verbs and the pipe operator for data wrangling
- 3 Download financial and macroeconomic data with **tidyfinance**
- 4 Compute and interpret logarithmic returns
- 5 Create time series plots and return distributions with **ggplot2**

Companion script

Open **Unit-1-TimeSeries-Foundations.R** — it contains worked examples followed by exercises you complete yourself.

What is a Time Series?

A **time series** is a sequence of observations $\{y_t\}_{t=1}^T$ recorded at successive points in time.

Distinguishing features:

- Observations are **ordered chronologically**
- Adjacent observations are typically **dependent**
- This dependence structure makes time series analysis distinct from cross-sectional methods

Examples

- Stock prices, exchange rates
- GDP, inflation (CPI)
- Interest rates, bond yields
- Temperature, rainfall

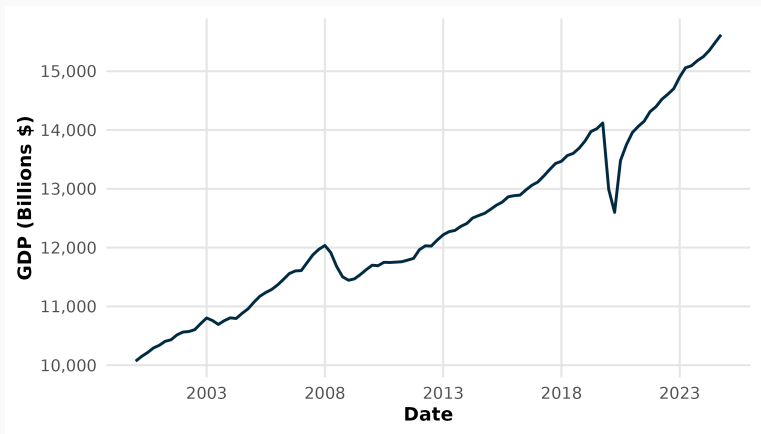
Any time series can be decomposed into structural components:

$$Y_t = T_t + S_t + C_t + \epsilon_t$$

Component	Symbol	Description
Trend	T_t	Long-term direction
Seasonality	S_t	Regular periodic patterns (e.g. quarterly)
Cyclical	C_t	Irregular longer-term fluctuations
Residual	ϵ_t	Random noise / innovation

In price-based financial time series the trend and residual components dominate; seasonality is often weak or absent. A multiplicative decomposition ($Y_t = T_t \times S_t \times C_t \times \epsilon_t$) is equivalent after taking logs.

What Does a Time Series Look Like?



A GDP-like series exhibits a clear upward **trend** (T_t) with short-run fluctuations ($C_t + \epsilon_t$) around it. Our goal: decompose, model and forecast such patterns.

Tidyverse Essentials

The **tidyverse** is a coherent collection of R packages for data science that share a common design philosophy.

Core packages:

- **dplyr** — data manipulation
- **ggplot2** — visualisation
- **tidyr** — reshaping data
- **readr** — reading data
- **tibble** — modern data frames
- **purrr** — functional programming

Key principles:

- Pipe operator `%>%` chains operations
- **Tidy data**: one observation per row, one variable per column
- Consistent, composable function API

Load everything at once
`library(tidyverse)`

Function	What it does
<code>filter()</code>	Keep rows that satisfy a condition
<code>select()</code>	Choose (or drop) columns by name
<code>mutate()</code>	Create new columns or transform existing ones
<code>summarise()</code>	Collapse rows into summary statistics
<code>group_by()</code>	Declare groups for subsequent operations
<code>arrange()</code>	Sort rows by one or more columns
<code>rename()</code>	Rename columns
<code>glimpse()</code>	Compact overview of a data frame

The Pipe Operator %>%

The pipe passes the left-hand result as the **first argument** to the right-hand function.



Without pipes: `summarise(group_by(filter(data, x > 0), cat), m = mean(y))`

With pipes: `data %>% filter(x > 0) %>% group_by(cat) %>% summarise(m = mean(y))`

Note: R 4.1+ also provides the native pipe `|>`; both work identically for our purposes.

Accessing Financial Data

The `tidyfinance` package provides tidy access to financial and economic data sources.

FRED macroeconomic data:

```
fred <- download_data("fred",  
  series = c("GDP", "CPIAUCNS"))
```

Stock price data:

```
aapl <- download_data(  
  "stock_prices",  
  symbols = "AAPL",  
  start = "2010-01-01",  
  end = "2020-01-01")
```

Key columns returned

- `date` — observation date
- `adjusted_close` — split/dividend-adjusted price
- `series / symbol`
- `value` (FRED) or `volume` (stocks)

Time Series Objects in R

A `tsibble` is a **time-series-aware** tibble (data frame).

Key features:

- Temporal **index** column
- Optional **key** columns for panel data
- `index_by()` for temporal aggregation
- Integrates with **fable** (modelling) and **feasts** (diagnostics)

Creating a `tsibble`:

```
data %>%  
  as_tsibble(  
    index = date,  
    regular = FALSE)
```

Temporal aggregation:

```
data %>%  
  index_by(m = ~yearmonth(.)) %>%  
  summarise(ret = sum(ret))
```

Log Returns & Visualisation

The **log return** (continuously compounded return) is:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln P_t - \ln P_{t-1}$$

Why log returns?

- ① Additivity — multi-period return = sum
- ② Symmetry — $\pm 10\%$ symmetric in log scale
- ③ Variance stabilisation
- ④ Closer to normality

Computing in R:

```
data %>% mutate(  
  lp = log(price),  
  lr = difference(lp))
```

Simple return

$$R_t = P_t/P_{t-1} - 1$$

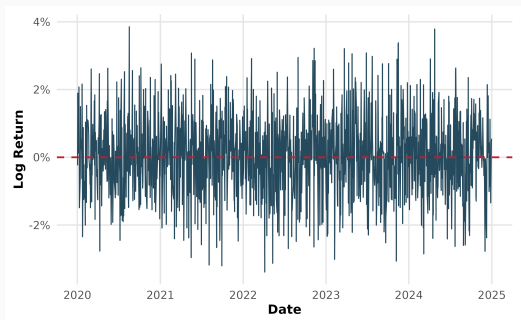
For small values: $r_t \approx R_t$

From Prices to Returns

Raw Prices

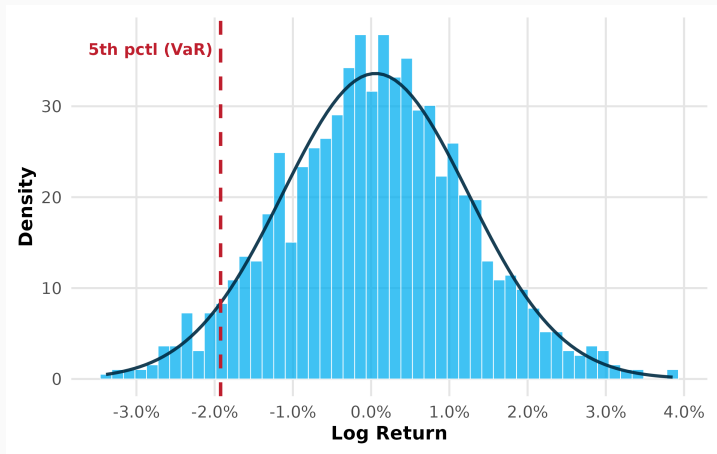


Log Returns



Prices exhibit a **trend** (non-stationary). Log returns fluctuate around zero (approximately stationary) but display **volatility clustering** — a preview of what we model in Unit 4.

Distribution of Returns



The dashed line marks the **5th percentile** — an informal measure of daily downside risk (Value at Risk). Note the **fat tails**: extreme returns occur more often than a normal distribution would predict.

Task	R code
Quick time series plot	<code>data %>% autoplot(.vars = price)</code>
Return histogram	<code>ggplot(aes(x = lreturn)) + geom_histogram(bins = 60)</code>
Reference line (VaR)	<code>+ geom_vline(xintercept = q05, linetype = "dashed")</code>
Percentage axis	<code>+ scale_x_continuous(labels = percent)</code>

Visualise early and often

Always plot your data before modelling. Visual inspection reveals trends, outliers and structural breaks that summary statistics miss.

Reshaping Data

`pivot_wider()`

Long → Wide (one column per series)

```
data %>% pivot_wider(  
  id_cols = year,  
  names_from = series,  
  values_from = value)
```

Use for cross-series comparison or when a function expects a matrix of returns (e.g. portfolio optimisation in Unit 5).

`pivot_longer()`

Wide → Long (tidy format)

```
data %>% pivot_longer(  
  cols = c(GDP, CPI),  
  names_to = "series",  
  values_to = "value")
```

Use for tidy analysis, faceted ggplots and group-wise summaries.

Exercises

Open `Unit-1-TimeSeries-Foundations.R` — the script walks you through a complete AAPL example before each exercise.

- ➊ **Macro data:** (a) Download GDP and CPI from FRED using `download_data("fred", ...)`; (b) compute yearly means with `index_by()` and `summarise()`; (c) pivot to wide format
- ➋ **Stock data:** Download a ticker of your choice (e.g. "MSFT", "TSLA") and convert to a `tsibble`
- ➌ **Log returns:** (a) Compute daily log returns with `mutate()` and `difference()`; (b) aggregate to weekly and monthly with `index_by()`; (c) plot all three with `autoplot()`
- ➍ **Distribution:** Plot a histogram of daily returns with `geom_histogram()` and add a 5th-percentile `geom_vline()`

Hints: `?download_data` | <https://fred.stlouisfed.org> | <https://uk.finance.yahoo.com>

You can now:

- Define a time series and identify its trend, seasonal and residual components
- Wrangle data fluently with **dplyr** verbs and the pipe operator
- Download financial and macroeconomic data via **tidyfinance**
- Store time series in a **tsibble** and aggregate across frequencies
- Compute log returns and explain why they are preferred over simple returns
- Visualise prices, returns and their distributions with **ggplot2**

Next: Unit 2 — Random Walks, Stationarity & Unit Root Tests